

CSE233 Week 10:

Command Line Arguments and File I/O

This week, we will discuss command line arguments and file I/O. Command line arguments allow data to be passed into a program when it is run. The command line arguments are an array of C-strings. Reading and writing files is a critical task for computer programs. This week, we'll discuss how to read and write files using the `std::ifstream` and `std::ofstream` types.

Command Line Arguments

Command line arguments are an array of C-strings that are provided to a program when run from the command line. Even desktop applications typically have the ability to accept command line arguments when they are run. Command line args are an important mechanism for passing data to a program. Most command line programs do not have interactive user input. Instead, they read all the data they expect from the user from the command line args.

To access them, you need to add two parameters to your main function:

```
int main(int argc, char **argv) {}  
// alternatively, you can do this for argv  
int main(int argc, char *argv[]) {}  
// or even this  
int main(int argc, char argv[][] ) {}
```

The argument, `argc` is the number of command line arguments passed to the program. The argument `argv` is an array of C-strings (and C-strings are also arrays, so it is an array of arrays) that contain the actual arguments. Every program has at least one command line argument, which is the command used to run the program itself. So, to access the

actual arguments to your program, you will need to start at the index of 1. Below is an example:

```
int main(int argc, char **argv) {  
  
    std::cout << "The name of the program is "  
        << argv[0] << std::endl;  
  
    std::cout << "The remaining arguments are: " << std::endl;  
    // start for loop at i = 1 to skip program name  
    for (int i = 1; i < argc; ++i) {  
        std::cout << argv[i] << std::endl;  
    }  
  
    return 0;  
}
```

Command line arguments are separated by spaces, so if we provided:

```
apple orange banana
```

as our command line args, and the name of the program was "fruit", then to run the program with those arguments we would do:

```
./fruit apple orange banana
```

in the terminal (many IDEs have ways to pass command line arguments for you).

The final output for this program would be:

```
The name of the program is `./fruit`  
The remaining command line arguments are:  
apple  
orange  
banana
```

File I/O

Reading and writing to files is the primary task for most computer programs. C++ provides the `fstream` header to read and write files.

The `std::ifstream` class is used to read files and the `std::ofstream`

class is used for writing files. These input and output streams provide the same operations as `std::cin` and `std::cout`, respectively. The difference is you first have to open a file for reading or writing.

Reading Files

To read a file, you need to declare an `std::ifstream` object. Files can be opened by passing a file path to the `ifstream` constructor, or by calling the `open()` function:

```
#include <fstream>
#include <iostream>
#include <string>

// reading file path from command line args
int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "Specify input file path as command line arg\n";
        return 1;
    }

    // open file with constructor
    std::ifstream fin(argv[1]);
    // alternative method:
    /*
    std::ifstream fin;
    fin.open(argv[1]);
    */

    // check that the file was opened correctly
    if (fin.is_open()) {
        // printing the contents of the file
        std::string input;
        // the >> operator works just like it does
        // for std::cin. it reads characters until
        // reaching a whitespace character
        // when reaching the end of the file, this
        // will return a "false-like" value
        while (fin >> input)
            std::cout << input << "\n";
    }
    else {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }

    // always close a file when you are done with it
    fin.close();
}
```

```
    return 0;  
}
```

If we assume the contents of the file we are reading look like this:

```
To be, or not to be, that is the question.  
2 + 2 = 4  
Lorem ipsum dolor...
```

Then the output of this program would be:

```
To  
be,  
or  
not  
to  
be,  
that  
is  
the  
question.  
2  
+  
2  
=  
4  
Lorem  
ipsum  
dolor...
```

Like `cin`, `ifstream` objects can also directly read numbers and other data types from a file. Let's assume we have a file that looks like this:

```
2 2  
4 4  
6 6  
8 8
```

We want to write a program that reads in the pairs of numbers and prints the result to the console.

```
#include <fstream>  
#include <iostream>  
  
// filestreams always need to be passed by reference
```

```

int readAndAdd(std::ifstream& fin) {
    int x;
    int y;
    int sum = 0;
    if (fin >> x >> y) {
        sum = x + y;
    }
    return sum;
}

int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "Specify input file path as command line arg\n";
        return 1;
    }

    // open file with constructor
    std::ifstream fin(argv[1]);

    // alternate method to check if file is open:
    if (fin.fail()) {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }

    // run the function readAndAdd() until the file is empty
    // fin.eof() returns true when the end of the file is reached
    while (!fin.eof()) {
        int sum = readAndAdd(fin);
        std::cout << "Sum = " << sum << "\n";
    }

    // close file
    fin.close();

    return 0;
}

```

There is the `std::getline()` function, which can read an entire line from a file into a string, preserving the whitespace. The first argument to `std::getline()` is an `ifstream` object, and the second argument is the `std::string` the line will be stored in.

```

#include <fstream>
#include <iostream>
#include <string>

int main(int argc, char** argv) {
    if (argc < 2) {

```

```

        std::cerr << "Specify input file path as command line arg\n";
        return 1;
    }

    // open file with constructor
    std::ifstream fin(argv[1]);

    if (fin.fail()) {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }

    // string variable to store text in
    std::string line;
    // loop through file and print each line
    while (std::getline(fin, line)) {
        std::cout << line << std::endl;
    }

    fin.close();
    return 0;
}

```

Writing Files

Creating and checking if an `std::ofstream` object is open is the same as it is for input files. Like `cout`, `ofstream` uses the insertion operator (`<<`) to write data to files. By default, output file streams truncate (i.e., delete) the contents of a file before writing to it. If the file does not exist, it will create the file. If you want to add data to an existing file, the `ios::app` option can be specified when opening the file. This flag is an option argument to the constructor and `open()` function. Below is an example program that copies one file's contents to another. Here, we are using another member function of `ifstream`, `get()`, which reads a single character at a time. The character that is read is passed by reference to the function.

```

#include <fstream>
#include <iostream>

int main(int argc, char** argv) {
    const char* errmsg = "Invalid usage. Expected 2 arguments\n";
    const char* help = "Usage: ./copy input_file output_file\n";

    if (argc < 3) {
        std::cerr << errmsg;
    }
}

```

```

        std::cerr << help;
    }

    std::ifstream fin(argv[1]);
    if (fin.fail()) {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }
    std::ofstream fout(argv[2]);
    if (fin.fail()) {
        // close fin before ending the program
        fin.close();
        std::cerr << "Failed to open file " << argv[2] << "\n";
        return 3;
    }

    char c;
    // while we are not at eof and can read a character
    while (!fin.eof() && fin.get(c)) {
        // print that character to the output file
        // this will create a character for character copy
        // of the input file, since get() does not skip whitespace
        fout << c;
    }

    fin.close();
    fout.close();

    return 0;
}

```

Conclusion

This week we discussed command line arguments and file I/O. Command line arguments are used to pass data into the program when it is run. Most console applications accept their user input primarily through command line arguments. Reading and writing files is the primary task of many computer programs. The `std::ifstream` class is used to read files, while `std::ofstream` is used to write files. It is important to check if the file objects are open before attempting to use them, and as well as to close them when you are done.